

FastAPI

The Complete Beginner's Guide

Build modern Python APIs, from your first line of code to deployment.

- 28 chapters in simple English
- 106 code examples, tested with FastAPI 0.157
- 3 complete projects: To-Do API, Library database, JWT login
- Validation, databases, security, testing, Docker deployment

Edition: October 2026

Contents

1. Introduction: What Is an API and What Is FastAPI? . . . 3
2. Python Basics You Need for FastAPI . . . 6
3. Installation and Setup . . . 8
4. Your First FastAPI Application . . . 10
5. HTTP Methods and Path Operations . . . 12
6. Path Parameters . . . 14
7. Query Parameters . . . 16
8. Request Body and Pydantic Models . . . 18
9. Validation in Depth . . . 21
10. Response Models and Status Codes . . . 24
11. Project 1: A Complete To-Do CRUD API . . . 27
12. Handling Errors . . . 30
13. Headers, Cookies, Forms and File Uploads . . . 32
14. Dependency Injection . . . 35
15. Async vs Sync: def or async def? . . . 38
16. Working with a Database (SQLModel) . . . 40
17. Authentication and Security (JWT) . . . 44
18. Structuring Bigger Applications (APIRouter) . . . 49
19. Middleware and CORS . . . 51
20. Background Tasks and Lifespan Events . . . 53
21. Settings and Environment Variables . . . 55
22. WebSockets, Static Files and Templates . . . 57
23. Customizing the API Documentation . . . 59
24. Testing Your API . . . 61
25. Deployment: Putting Your API Online . . . 63
26. Best Practices and Common Mistakes . . . 65
27. Cheat Sheet . . . 67
28. Glossary and What to Learn Next . . . 69

CHAPTER 1

1. Introduction: What Is an API and What Is FastAPI?

Welcome! This guide teaches you FastAPI from zero to deploying a real application. You do not need any web development experience. If you know a little Python (variables, functions, lists and dictionaries), you are ready.

What is an API?

API stands for **Application Programming Interface**. That sounds complicated, but the idea is simple: an API is a way for one program to talk to another program.

Think of a **restaurant**:

- **You (the customer)** are the *client*: a mobile app, a website, or another program.
- **The kitchen** is the *server*: it has the data and does the work.
- **The waiter** is the *API*: it takes your order to the kitchen and brings the food back.
- **The menu** is the *API documentation*: it tells you what you are allowed to order.

You never walk into the kitchen yourself. You give a clear order to the waiter, and the waiter returns exactly what you asked for. A web API works the same way: a client sends a **request**, the server sends back a **response**.

How web APIs talk: HTTP and JSON

Web APIs use **HTTP**, the same language your browser uses to load websites. Every request has:

- **A URL** (the address), for example `https://api.example.com/books/42`
- **A method** (the action), for example `GET` to read or `POST` to create
- **Optional data** (the body), usually written in **JSON**

JSON (JavaScript Object Notation) is a simple text format for data. It looks almost exactly like a Python dictionary:

JSON

```
{
  "id": 42,
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "available": true,
  "tags": ["programming", "best-practices"]
}
```

What is FastAPI?

FastAPI is a modern Python framework for building APIs. It was created by Sebastian Ramirez (known online as *tiangolo*) and is now one of the most popular Python web frameworks in the world. It is built on two excellent libraries:

- **Starlette**: handles the web part (receiving requests, sending responses, WebSockets).

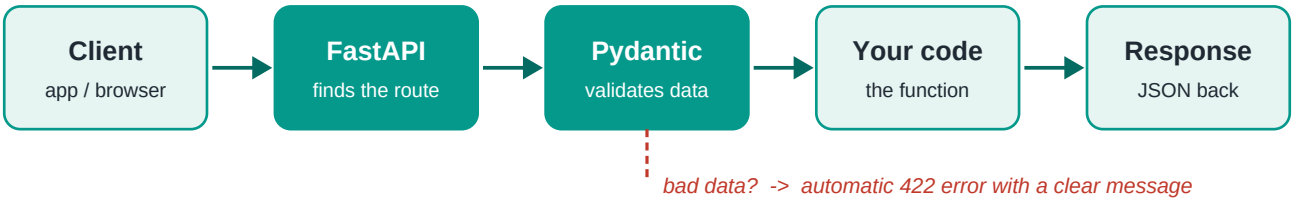
- **Pydantic**: handles the data part (checking that data is correct and converting it).

Why do people love FastAPI?

Feature	What it means for you
Fast to run	One of the fastest Python frameworks, comparable to Node.js and Go for many workloads.
Fast to write	You write less code. Type hints do most of the work for you.
Automatic validation	Wrong data is rejected automatically with a clear error message.
Automatic documentation	Interactive docs pages (/docs and /redoc) are generated for free.
Editor support	Autocomplete and error checking everywhere, because everything has types.
Standards-based	Built on OpenAPI and JSON Schema, open standards that many tools understand.
Async support	Can handle many users at the same time efficiently.

How a request flows through FastAPI

This picture shows what happens when a client calls your API. Keep it in mind; every chapter fills in one of these boxes.



1. The **client** sends an HTTP request (for example `POST /books` with JSON data).
2. **FastAPI** finds the function (called a *path operation*) that matches the URL and method.
3. **Pydantic** checks the data. If it is wrong, FastAPI immediately replies with error `422` and a helpful message.
4. **Your function** runs with clean, correct Python data.
5. FastAPI converts what you return into **JSON** and sends the response back.

How to use this guide

Each chapter builds on the previous one. Type the code yourself instead of copying it; you will learn much faster. Look for these boxes:

> **TIP** Tip boxes give helpful shortcuts and good habits.

> **NOTE** Note boxes explain something extra or a detail worth knowing.

WARNING Warning boxes point out common mistakes that beginners make.

NOTE All examples in this guide were tested with FastAPI 0.157 on Python 3.13 (October 2026). FastAPI changes often, but the basics in this guide are stable. If something looks different in a newer version, check the official docs at fastapi.tiangolo.com.

CHAPTER 2

2. Python Basics You Need for FastAPI

FastAPI uses a few Python features that beginners may not have seen yet. Let's learn them first. This chapter will make everything later much easier.

Type hints

A **type hint** tells Python (and your editor, and FastAPI) what kind of value a variable should hold. You write a colon and the type after the name:

```python
name: str = "Nirodha"
age: int = 25
price: float = 9.99
is_active: bool = True

def greet(name: str) -> str:      # "-> str" means: this function returns a string
    return "Hello " + name
```

In normal Python, type hints are only notes; Python does not enforce them. **In FastAPI they are powerful**: FastAPI reads them to know how to validate data, convert it, and document it.

Common types

Type hint	Meaning	Example value
`str`	Text	`"hello"`
`int`	Whole number	`42`
`float`	Decimal number	`3.14`
`bool`	True or False	`True`
`list[str]`	A list of strings	`["a", "b"]`
`dict[str, int]`	A dictionary: string keys, int values	`{"apples": 3}`
`str | None`	A string OR nothing (optional)	`"hi"` or `None`
`datetime`	A date and time (import from datetime)	`datetime(2026, 1, 1)`

NOTE `str | None` means "this can be a string or None". You will see it a lot for optional values. Older code writes the same thing as `Optional[str]` (from the typing module). Both mean the same.

Annotated: adding extra information to a type

`Annotated` lets you attach extra information to a type hint. FastAPI uses it to add rules (like "at least 3 characters"). The first part is the type, the rest is extra information:

```python
from typing import Annotated
from fastapi import Query
```

```
# "q is a string, and here are extra FastAPI rules for it"
q: Annotated[str, Query(min_length=3)]
```

Don't worry if this feels strange now. You will use it many times and it will become natural.

Decorators

A **decorator** is the line starting with `@` above a function. It "wraps" the function to give it extra behaviour. In FastAPI, a decorator connects a URL to your function:

PYTHON

```
@app.get("/hello")        # "When someone does GET /hello ...
def say_hello():          #  ... run this function"
    return {"message": "Hello"}
```

async and await

You will see functions written as `async def` instead of `def`. An **async** function can *pause* while it waits for something slow (like a network call or a database) and let Python do other work in the meantime. Inside an async function you use `await` to wait for slow things.

PYTHON

```
async def get_data():
    result = await some_slow_network_call()   # pause here, let others work
    return result
```

Chapter 15 explains exactly when to use `async def` and when to use plain `def`. For now, just know both work in FastAPI.

Classes (for data models)

FastAPI uses **classes** to describe the shape of data. You only need the simplest form:

PYTHON

```
from pydantic import BaseModel

class Book(BaseModel):     # a "model": the shape of a book
    title: str
    author: str
    pages: int

book = Book(title="Dune", author="Frank Herbert", pages=412)
print(book.title)          # Dune
```

CHAPTER 3

3. Installation and Setup

Step 1: Install Python

You need **Python 3.10 or newer** (this guide uses the modern `str | None` syntax). Check your version:

```
TERMINAL
$ python --version        # Windows
$ python3 --version       # macOS / Linux
```

If you do not have Python, download it from **python.org**. On Windows, tick the box **"Add Python to PATH"** during installation.

Step 2: Create a project folder

```
TERMINAL
$ mkdir my-first-api
$ cd my-first-api
```

Step 3: Create a virtual environment

A **virtual environment** is a private box for your project's libraries, so different projects do not interfere with each other. Always use one.

```
TERMINAL
# Create it (do this once)
$ python -m venv .venv

# Activate it (do this every time you open a new terminal)
$ .venv\Scripts\activate          # Windows (Command Prompt / PowerShell)
$ source .venv/bin/activate        # macOS / Linux
```

When it is active, you will see `(.venv)` at the start of your terminal line.

Step 4: Install FastAPI

```
TERMINAL
$ pip install "fastapi[standard]"
```

The `[standard]` part installs FastAPI **plus** the most useful extras:

Package	What it does
`uvicorn`	The web server that actually runs your app
`fastapi-cli`	Gives you the `fastapi dev` and `fastapi run` commands
`httpx`	Needed for testing (and for calling other APIs)
`python-multipart`	Needed for HTML forms and file uploads

Package	What it does
`email-validator`	Lets you validate email addresses
`jinja2`	HTML templates

WARNING Keep the quotes around "fastapi[standard]". Some terminals (like zsh on macOS) give an error without them.

Step 5: Save your requirements

It is good practice to save the list of libraries your project needs, so others (or a server) can install the same ones:

TERMINAL

```
$ pip freeze > requirements.txt          # save
$ pip install -r requirements.txt        # install them somewhere else later
```

Recommended editor

Use **Visual Studio Code** (free) with the **Python** extension, or **PyCharm**. Both give you autocomplete and show errors as you type, and FastAPI is designed to make this work very well.

TIP Modern alternative: the tool `uv` (by Astral) can create environments and install packages very quickly. Commands like `uv init`, `uv add "fastapi[standard]"` and `uv run fastapi dev` do the same jobs as above. It is optional; plain pip works perfectly.

CHAPTER 4

4. Your First FastAPI Application

Write the code

Create a file called **main.py** in your project folder:

```python
from fastapi import FastAPI      # 1. import FastAPI

app = FastAPI()                  # 2. create the application


@app.get("/")                    # 3. connect GET "/" to the function below
def read_root():                 # 4. the function that runs
    return {"message": "Hello, World!"}   # 5. return data (becomes JSON)
```

That's a complete API in 5 lines! Let's understand each line:

1. We import the `FastAPI` class.
2. We create an **app** object. This is the heart of your API; everything attaches to it.
3. `@app.get("/")` is a **path operation decorator**. It says: when a `GET` request arrives at path `/`, call the function below.
4. `read_root` is the **path operation function**. You can name it anything.
5. We return a Python dictionary. FastAPI automatically turns it into JSON.

Run the server

```
$ fastapi dev main.py
```

You will see output telling you the server is running at **http://127.0.0.1:8000**. The `dev` mode **auto-reloads**: every time you save your file, the server restarts automatically.

Open your browser and go to http://127.0.0.1:8000. You will see:

```json
{"message": "Hello, World!"}
```

TIP To stop the server, press Ctrl + C in the terminal.

NOTE `127.0.0.1` (also called `localhost`) means "this computer". `8000` is the **port**, like a door number. Only you can see the server right now; it is not on the internet.

You can also run the server with Uvicorn directly. This does the same thing:

```
$ uvicorn main:app --reload
# "main" = the file main.py, "app" = the variable inside it
```

The magic: automatic interactive documentation

Now open **http://127.0.0.1:8000/docs**. You will see a beautiful page called **Swagger UI** listing every endpoint in your API. You did not write any of it!

1. Click on the `GET /` row to expand it.
2. Click **Try it out**, then **Execute**.
3. You will see the real response from your server, the status code, and even the equivalent `curl` command.

There is a second documentation style at **http://127.0.0.1:8000/redoc** (ReDoc), which is nicer for reading. And the raw machine-readable description of your API is at **/openapi.json**.

> **TIP** The /docs page is the best way to test your API while learning. You never need extra tools like Postman (though you can use them if you like).

Add a second endpoint

PYTHON

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def read_root():
    return {"message": "Hello, World!"}


@app.get("/about")
def about():
    return {"app": "My First API", "version": "1.0", "author": "Me"}
```

Save the file. The server reloads automatically. Visit /about or refresh /docs to see it.

What can you return?

A path operation function can return almost anything, and FastAPI converts it to JSON:

- `dict`, `list`, `str`, `int`, `float`, `bool`, `None`
- Pydantic models (Chapter 8)
- Database objects (Chapter 16)
- Special responses like HTML, files, or redirects (Chapter 10)

CHAPTER 5

5. HTTP Methods and Path Operations

In FastAPI language, a **path** is the part of the URL after the domain (like `/books`), and an **operation** is the HTTP method. Together they form a **path operation**, also called an **endpoint**.

The main HTTP methods

Method	Purpose	Example	FastAPI decorator
GET	Read data	Get a list of books	`@app.get()`
POST	Create new data	Add a new book	`@app.post()`
PUT	Replace data completely	Replace a whole book record	`@app.put()`
PATCH	Update part of the data	Change only the price	`@app.patch()`
DELETE	Remove data	Delete a book	`@app.delete()`

These map to the four basic data actions known as **CRUD**: **C**reate (POST), **R**ead (GET), **U**pdate (PUT/PATCH) and **D**elete (DELETE).

PYTHON
```
from fastapi import FastAPI

app = FastAPI()


@app.get("/books")
def list_books():
    return {"action": "list all books"}


@app.post("/books")
def create_book():
    return {"action": "create a book"}


@app.put("/books/{book_id}")
def replace_book(book_id: int):
    return {"action": "replace", "id": book_id}


@app.patch("/books/{book_id}")
def update_book(book_id: int):
    return {"action": "update", "id": book_id}


@app.delete("/books/{book_id}")
def delete_book(book_id: int):
    return {"action": "delete", "id": book_id}
```

NOTE Your browser's address bar can only send GET requests. To try POST, PUT, PATCH and DELETE, use the /docs page.

Good URL design (REST conventions)

- Use **nouns**, not verbs: `/books`, not `/getBooks`. The method already says the action.
- Use **plural** names for collections: `/books`, `/users`.
- Put the ID in the path to point at one item: `/books/42`.
- Nest when there is a clear relationship: `/users/7/orders`.
- Use lowercase and hyphens: `/book-reviews`, not `/BookReviews`.

CHAPTER 6

6. Path Parameters

A **path parameter** is a variable part of the URL. You write it in curly braces `{}` in the path, and add a function argument with the **same name**.

```python
@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"book_id": book_id}
```

Now `/books/5` returns `{"book_id": 5}`. Notice the type hint `int`. It gives you two things for free:

- **Conversion**: the URL is text, but FastAPI converts `"5"` into the integer `5`.
- **Validation**: if someone visits `/books/abc`, FastAPI automatically returns a `422` error explaining that an integer was expected. Your function never even runs.

```json
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "book_id"],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "abc"
    }
  ]
}
```

Multiple path parameters

```python
@app.get("/users/{user_id}/orders/{order_id}")
def get_user_order(user_id: int, order_id: int):
    return {"user_id": user_id, "order_id": order_id}
```

Order matters!

FastAPI checks paths **from top to bottom** and uses the first match. A fixed path must come **before** a path with a parameter that could match it:

```python
@app.get("/users/me")            # must be FIRST
def read_current_user():
    return {"user": "the current user"}


@app.get("/users/{user_id}")     # otherwise this would catch "/users/me"
def read_user(user_id: str):
    return {"user_id": user_id}
```

WARNING If you put /users/{user_id} first, visiting /users/me would call read_user with user_id = "me".

Limiting values with Enum

If a parameter should only accept a few fixed values, use an **Enum**. The docs page will even show a dropdown:

PYTHON

```
from enum import Enum
from fastapi import FastAPI

app = FastAPI()


class Category(str, Enum):
    fiction = "fiction"
    science = "science"
    history = "history"


@app.get("/categories/{category}")
def books_in_category(category: Category):
    if category == Category.science:
        return {"category": category, "note": "Great choice for curious minds!"}
    return {"category": category}
```

Visiting `/categories/cooking` returns a `422` error listing the allowed values.

Paths that contain slashes

If a parameter must hold a whole file path (with `/` inside), add `:path`:

PYTHON

```
@app.get("/files/{file_path:path}")
def read_file(file_path: str):
    return {"file_path": file_path}     # /files/home/me/a.txt -> "home/me/a.txt"
```

CHAPTER 7

7. Query Parameters

Query parameters come after a `?` in the URL, separated by `&`. For example: `/books?skip=0&limit=10`. They are great for **optional** things like filtering, searching, sorting and pagination.

In FastAPI, **any function argument that is not in the path is a query parameter**. That's it!

```python
from fastapi import FastAPI

app = FastAPI()

fake_books = [{"title": f"Book {i}"} for i in range(1, 101)]   # 100 fake books


@app.get("/books")
def list_books(skip: int = 0, limit: int = 10):
    return fake_books[skip : skip + limit]
```

- `/books` returns books 1 to 10 (using the defaults `skip=0`, `limit=10`)
- `/books?skip=20&limit=5` returns books 21 to 25
- `/books?limit=abc` returns a `422` error, because `limit` must be an integer

Required, optional and default values

How you write it	What it means
`q: str`	**Required.** Missing it gives a 422 error.
`limit: int = 10`	**Optional** with a default value of 10.
`q: str | None = None`	**Optional.** If not sent, the value is None.

```python
@app.get("/search")
def search(keyword: str, page: int = 1, category: str | None = None):
    result = {"keyword": keyword, "page": page}
    if category:
        result["category"] = category
    return result
```

Boolean query parameters

FastAPI is clever with `bool`. All of these become `True`: `?available=true`, `?available=1`, `?available=yes`, `?available=on`. And `false`, `0`, `no`, `off` become `False`.

```python
@app.get("/products")
def list_products(in_stock: bool = False):
    return {"only_in_stock": in_stock}
```

Mixing path and query parameters

FastAPI knows which is which by looking at the path string. No special order is needed:

PYTHON

```python
@app.get("/users/{user_id}/books")
def user_books(user_id: int, status: str = "reading", limit: int = 20):
    return {"user_id": user_id, "status": status, "limit": limit}
# /users/3/books?status=finished
```

Lists in query parameters

To accept the same parameter many times (like `?tag=python&tag=web`), use a list with `Query()`:

PYTHON

```python
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/articles")
def filter_articles(tag: Annotated[list[str], Query()] = []):
    return {"tags": tag}       # /articles?tag=python&tag=web -> ["python", "web"]
```

CHAPTER 8

8. Request Body and Pydantic Models

Path and query parameters are good for small values. But when a client wants to **send real data** (like a new user with a name, email and password), it sends it in the **request body**, as JSON. This is usually done with `POST`, `PUT` or `PATCH`.

Step 1: Describe the data with a Pydantic model

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Book(BaseModel):
    title: str                      # required
    author: str                     # required
    pages: int                      # required
    description: str | None = None  # optional
    price: float = 0.0              # optional, default 0.0
```

A **model** is a class that inherits from `BaseModel`. Each attribute is a field. Fields **without** a default are required; fields **with** a default are optional.

Step 2: Use the model as a parameter

```python
@app.post("/books")
def create_book(book: Book):
    print(book.title)               # use it like a normal Python object
    return {"message": "Book created", "book": book}
```

Because the parameter type is a Pydantic model, FastAPI knows it must come from the **body**. Now FastAPI will:

1. Read the body of the request as JSON.
2. Convert values to the right types (for example `"412"` to `412` where possible).
3. Validate the data. If it is wrong, return a clear `422` error explaining exactly which field is wrong.
4. Give your function a ready-to-use `book` object with autocomplete in your editor.
5. Add the model's JSON Schema to the /docs page.

A client would send something like this:

```json
{
  "title": "Dune",
  "author": "Frank Herbert",
  "pages": 412
}
```

Working with the model

```python
@app.post("/books/with-tax")
def create_book_with_tax(book: Book):
    data = book.model_dump()                    # convert the model to a dict
    data["price_with_tax"] = round(book.price * 1.18, 2)
    return data
```

Pydantic method	What it does
`model.model_dump()`	Convert to a Python dictionary
`model.model_dump(exclude_unset=True)`	Only fields the client actually sent (great for PATCH)
`model.model_dump_json()`	Convert to a JSON string
`Model.model_validate(data)`	Create a model from a dict or another object (with validation)
`model.model_copy(update={...})`	Copy the model, changing some fields

NOTE These are **Pydantic version 2** method names. Old tutorials use `.dict()` and `.json()`; those are from Pydantic v1 and are deprecated.

Body + path + query together

You can use all three at once. FastAPI decides where each comes from:

- If the name is in the **path** string, it is a path parameter.
- If the type is a **Pydantic model**, it is the request body.
- If it is a simple type (`int`, `str`, ...) and not in the path, it is a **query** parameter.

```python
@app.put("/books/{book_id}")
def replace_book(book_id: int, book: Book, notify: bool = False):
    return {"book_id": book_id, "book": book, "notify": notify}
```

Nested models

Models can contain other models and lists of models. This lets you describe complex JSON easily:

```python
from pydantic import BaseModel, HttpUrl


class Image(BaseModel):
    url: HttpUrl                # must be a valid http/https URL
    caption: str | None = None


class Author(BaseModel):
    name: str
    country: str | None = None
```

```
class Book(BaseModel):
    title: str
    author: Author              # a model inside a model
    tags: list[str] = []        # a list of strings
    images: list[Image] = []    # a list of models
```

JSON

```
{
  "title": "Dune",
  "author": {"name": "Frank Herbert", "country": "USA"},
  "tags": ["sci-fi", "classic"],
  "images": [{"url": "https://example.com/dune.jpg", "caption": "Cover"}]
}
```

Special field types

Type	Import from	Checks that the value is...
`EmailStr`	pydantic	a valid email address
`HttpUrl`	pydantic	a valid web URL
`datetime`, `date`, `time`	datetime	a date/time (accepts ISO text like 2026-10-21)
`UUID`	uuid	a valid UUID
`Decimal`	decimal	an exact decimal number (good for money)
`Literal["a", "b"]`	typing	exactly one of the listed values

Adding examples to the docs

You can show an example request on the /docs page so users know what to send:

PYTHON

```
class Book(BaseModel):
    title: str
    author: str
    pages: int

    model_config = {
        "json_schema_extra": {
            "examples": [
                {"title": "Dune", "author": "Frank Herbert", "pages": 412}
            ]
        }
    }
```

CHAPTER 9

9. Validation in Depth

Validation means **checking that data follows your rules** before you use it. FastAPI makes this very easy and you should use it everywhere. Good validation protects your app from bad data and attackers.

Validating query parameters with Query()

PYTHON

```
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()


@app.get("/search")
def search(
    q: Annotated[str, Query(min_length=3, max_length=50)],
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=100)] = 20,
):
    return {"q": q, "page": page, "size": size}
```

Validating path parameters with Path()

PYTHON

```
from typing import Annotated
from fastapi import Path


@app.get("/books/{book_id}")
def get_book(book_id: Annotated[int, Path(title="The book ID", gt=0)]):
    return {"book_id": book_id}
```

Validating body fields with Field()

PYTHON

```
from pydantic import BaseModel, EmailStr, Field


class UserCreate(BaseModel):
    username: str = Field(min_length=3, max_length=20, pattern=r"^[a-zA-Z0-9_]+$")
    email: EmailStr
    age: int = Field(ge=13, le=120, description="Must be 13 or older")
    bio: str | None = Field(default=None, max_length=300)
```

All the common rules

Rule	Works on	Meaning
`gt=0`	numbers	greater than
`ge=1`	numbers	greater than or equal

Rule	Works on	Meaning
`lt=100`	numbers	less than
`le=100`	numbers	less than or equal
`multiple_of=5`	numbers	must be divisible by 5
`min_length=3`	strings, lists	minimum length
`max_length=50`	strings, lists	maximum length
`pattern=r"..."`	strings	must match a regular expression
`alias="item-query"`	Query/Path/Header	use a different name in the URL
`deprecated=True`	Query/Path/Field	mark as deprecated in the docs
`title`, `description`	all	documentation text shown in /docs

Custom validation rules

When the built-in rules are not enough, write your own with `@field_validator` (one field) or `@model_validator` (several fields together):

PYTHON

```
from pydantic import BaseModel, field_validator, model_validator


class SignUp(BaseModel):
    username: str
    password: str
    confirm_password: str

    @field_validator("username")
    @classmethod
    def no_spaces(cls, value: str) -> str:
        if " " in value:
            raise ValueError("username must not contain spaces")
        return value.lower()            # you can also clean the value

    @model_validator(mode="after")
    def passwords_match(self):
        if self.password != self.confirm_password:
            raise ValueError("passwords do not match")
        return self
```

When a validator raises `ValueError`, FastAPI returns a `422` error with your message. No extra code needed.

Understanding the 422 error

Every validation error has the same shape, so clients can show helpful messages:

Key	Meaning
`loc`	Where the error is, e.g. `["body", "age"]` or `["query", "page"]`
`msg`	A human-readable explanation

Key	Meaning
`type`	A machine-readable error code, e.g. `greater_than_equal`
`input`	The value the client actually sent

CHAPTER 10

10. Response Models and Status Codes

Why you need a response model

Imagine a user model that includes a password. When you create a user, you must **never** send the password back. A **response model** tells FastAPI exactly which fields are allowed in the response. Anything else is removed automatically.

PYTHON

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()


class UserIn(BaseModel):          # what the client SENDS
    username: str
    email: EmailStr
    password: str


class UserOut(BaseModel):         # what we SEND BACK (no password!)
    username: str
    email: EmailStr


@app.post("/users", response_model=UserOut)
def create_user(user: UserIn):
    # ... save the user (with a hashed password) ...
    return user                   # FastAPI filters out "password" for you
```

A response model also: validates your output, converts it to JSON, and documents the response in /docs.

Two ways to declare it

PYTHON

```
# Way 1: the response_model argument
@app.get("/users/{user_id}", response_model=UserOut)
def get_user(user_id: int):
    ...

# Way 2: the return type annotation (the modern, shorter way)
@app.get("/users/{user_id}")
def get_user2(user_id: int) -> UserOut:
    ...
```

TIP Use the return annotation (Way 2) when you return exactly that type. Use `response_model` when you return something different (like a database object or a model with extra fields) that should be filtered.

Useful response model options

```python
@app.get("/books/{book_id}", response_model=Book, response_model_exclude_unset=True)
def get_book(book_id: int):
    ...   # fields that were never set are left out of the response
```

HTTP status codes

Every response has a **status code**: a number that says how the request went. Learn these:

Code	Name	When to use it
200	OK	Success (the default for GET, PUT, PATCH)
201	Created	A new item was created (use for POST)
204	No Content	Success, nothing to send back (often for DELETE)
400	Bad Request	The request makes no sense for business reasons
401	Unauthorized	Not logged in / bad or missing token
403	Forbidden	Logged in, but not allowed to do this
404	Not Found	The item does not exist
409	Conflict	Clash with existing data, e.g. username already taken
422	Unprocessable Content	Validation failed (FastAPI sends this automatically)
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	A bug or crash on the server

Remember the families: **2xx** success, **3xx** redirect, **4xx** the client made a mistake, **5xx** the server made a mistake.

```python
from fastapi import FastAPI, status

app = FastAPI()


@app.post("/books", status_code=201)
def create_book(book: Book):
    return book


@app.delete("/books/{book_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_book(book_id: int):
    return None
```

TIP `status.HTTP_201_CREATED` is the same as `201`, but easier to read and your editor autocompletes it.

Other response types

By default FastAPI returns JSON. You can return other kinds of responses too:

PYTHON

```
from fastapi.responses import (
    FileResponse, HTMLResponse, JSONResponse, PlainTextResponse,
    RedirectResponse, StreamingResponse,
)


@app.get("/hello-html", response_class=HTMLResponse)
def hello_html():
    return "<h1>Hello from FastAPI</h1>"


@app.get("/download")
def download():
    return FileResponse("report.pdf", filename="report.pdf")


@app.get("/old-page")
def old_page():
    return RedirectResponse(url="/new-page")


@app.get("/custom")
def custom():
    return JSONResponse(status_code=202, content={"status": "accepted"},
                        headers={"X-Custom": "hello"})
```

CHAPTER 11

11. Project 1: A Complete To-Do CRUD API

Time to combine everything into a real, working project. This to-do API stores data in memory (a dictionary) so you can focus on FastAPI. In Chapter 16 we will use a real database.

Create **main.py** with this code:

PYTHON

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(title="To-Do API", version="1.0.0")


# ---------- Models (the shape of our data) ----------
class TodoCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: str | None = None
    done: bool = False


class TodoUpdate(BaseModel):
    title: str | None = Field(default=None, min_length=1, max_length=100)
    description: str | None = None
    done: bool | None = None


class Todo(TodoCreate):
    id: int


# ---------- "Database" (just a dictionary in memory) ----------
todos: dict[int, Todo] = {}
next_id = 1


# ---------- Endpoints ----------
@app.post("/todos", response_model=Todo, status_code=status.HTTP_201_CREATED)
def create_todo(todo: TodoCreate):
    global next_id
    new_todo = Todo(id=next_id, **todo.model_dump())
    todos[next_id] = new_todo
    next_id += 1
    return new_todo


@app.get("/todos", response_model=list[Todo])
def list_todos(done: bool | None = None, skip: int = 0, limit: int = 10):
    items = list(todos.values())
    if done is not None:                      # optional filter
        items = [t for t in items if t.done == done]
    return items[skip : skip + limit]         # simple pagination


@app.get("/todos/{todo_id}", response_model=Todo)
def get_todo(todo_id: int):
    if todo_id not in todos:
        raise HTTPException(status_code=404, detail="Todo not found")
    return todos[todo_id]
```

```python
@app.patch("/todos/{todo_id}", response_model=Todo)
def update_todo(todo_id: int, changes: TodoUpdate):
    if todo_id not in todos:
        raise HTTPException(status_code=404, detail="Todo not found")
    stored = todos[todo_id]
    update_data = changes.model_dump(exclude_unset=True)  # only sent fields
    updated = stored.model_copy(update=update_data)
    todos[todo_id] = updated
    return updated


@app.delete("/todos/{todo_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_todo(todo_id: int):
    if todo_id not in todos:
        raise HTTPException(status_code=404, detail="Todo not found")
    del todos[todo_id]
```

Run and try it

TERMINAL

```
$ fastapi dev main.py
```

Open http://127.0.0.1:8000/docs and try these steps:

1. **POST /todos** with `{"title": "Buy milk"}`. You get status `201` and the new to-do with `id: 1`.
2. **GET /todos** to see the list. Try `?done=false`.
3. **PATCH /todos/1** with `{"done": true}` to mark it finished. Only `done` changes.
4. **GET /todos/99** returns `404` with `{"detail": "Todo not found"}`.
5. **POST /todos** with `{"title": ""}` returns `422` because the title is too short.
6. **DELETE /todos/1** returns `204`.

What you just used

Concept	Where in the code
Pydantic models with validation	`TodoCreate`, `TodoUpdate`, `Field(min_length=1)`
Model inheritance	`Todo(TodoCreate)` adds an `id` to the create fields
Request body	`todo: TodoCreate`, `changes: TodoUpdate`
Path parameters	`/todos/{todo_id}`
Optional query parameters	`done`, `skip`, `limit` in `list_todos`
Response models	`response_model=Todo`, `response_model=list[Todo]`
Status codes	`201` for create, `204` for delete
Error handling	`HTTPException(status_code=404)`
Partial update	`model_dump(exclude_unset=True)` + `model_copy(update=...)`

WARNING Data in memory disappears every time the server restarts. That's fine for learning; real apps use a database.

CHAPTER 12

12. Handling Errors

HTTPException

When something goes wrong (item not found, no permission...), **raise** an `HTTPException`. FastAPI stops the function and sends a proper error response to the client:

PYTHON

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
books = {1: "Dune", 2: "Emma"}


@app.get("/books/{book_id}")
def get_book(book_id: int):
    if book_id not in books:
        raise HTTPException(
            status_code=404,
            detail="Book not found",
            headers={"X-Error": "missing-book"},   # optional
        )
    return {"title": books[book_id]}
```

The client receives status `404` and the body `{"detail": "Book not found"}`. The `detail` can also be a dict or list if you want to send more information.

> **NOTE** You use `raise`, not `return`. Raising works even from deep inside helper functions and dependencies, which makes it very convenient.

Your own exceptions and handlers

For bigger apps, create your own exception classes and tell FastAPI how to turn them into responses:

PYTHON

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()


class OutOfStockError(Exception):
    def __init__(self, product: str):
        self.product = product


@app.exception_handler(OutOfStockError)
async def out_of_stock_handler(request: Request, exc: OutOfStockError):
    return JSONResponse(
        status_code=409,
        content={"error": "out_of_stock", "message": f"{exc.product} is sold out"},
    )
```

```
@app.post("/orders/{product}")
def place_order(product: str):
    if product == "iphone":
        raise OutOfStockError(product)     # handled by the function above
    return {"ordered": product}
```

Customizing validation errors

You can change how the automatic `422` errors look, for example to match your company's format:

PYTHON

```
from fastapi.exceptions import RequestValidationError


@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc: RequestValidationError):
    errors = [
        {"field": ".".join(str(p) for p in e["loc"][1:]), "message": e["msg"]}
        for e in exc.errors()
    ]
    return JSONResponse(status_code=422, content={"errors": errors})
```

Catching unexpected errors

PYTHON

```
import logging

logger = logging.getLogger("app")


@app.exception_handler(Exception)
async def unexpected_error_handler(request: Request, exc: Exception):
    logger.exception("Unexpected error")          # log full details for yourself
    return JSONResponse(status_code=500,
                        content={"detail": "Something went wrong. Please try again."})
```

WARNING Never send internal error details (stack traces, SQL, file paths) to the client. Log them on the server instead.

CHAPTER 13

13. Headers, Cookies, Forms and File Uploads

Reading headers

Headers are extra information sent with a request (browser type, tokens, language...). Use `Header()`:

PYTHON
```
from typing import Annotated
from fastapi import FastAPI, Header

app = FastAPI()


@app.get("/info")
def info(
    user_agent: Annotated[str | None, Header()] = None,
    accept_language: Annotated[str | None, Header()] = None,
    x_request_id: Annotated[str | None, Header()] = None,
):
    return {"user_agent": user_agent, "language": accept_language, "id": x_request_id}
```

NOTE Header names use hyphens (`User-Agent`), but Python names can't. FastAPI converts underscores to hyphens automatically, and header names are not case-sensitive.

Reading and setting cookies

PYTHON
```
from fastapi import Cookie, Response


@app.get("/set-theme")
def set_theme(response: Response, theme: str = "dark"):
    response.set_cookie(key="theme", value=theme, max_age=3600, httponly=True)
    return {"message": f"theme set to {theme}"}


@app.get("/get-theme")
def get_theme(theme: Annotated[str | None, Cookie()] = None):
    return {"theme": theme}
```

HTML form data

HTML forms send data in a different format from JSON. Use `Form()` for it (requires `python-multipart`, included in `fastapi[standard]`):

PYTHON
```
from fastapi import Form


@app.post("/login-form")
def login_form(
    username: Annotated[str, Form()],
```

```
    password: Annotated[str, Form()],
):
    return {"username": username}
```

You can also group form fields into a Pydantic model:

PYTHON

```
class ContactForm(BaseModel):
    name: str
    email: EmailStr
    message: str


@app.post("/contact")
def contact(data: Annotated[ContactForm, Form()]):
    return {"received": data}
```

WARNING A single endpoint cannot receive both JSON body and form fields. This is a rule of HTTP, not of FastAPI.

File uploads

PYTHON

```
from fastapi import File, UploadFile, HTTPException

ALLOWED_TYPES = {"image/png", "image/jpeg"}
MAX_SIZE = 2 * 1024 * 1024    # 2 MB


@app.post("/upload")
async def upload_image(file: UploadFile):
    if file.content_type not in ALLOWED_TYPES:
        raise HTTPException(400, "Only PNG and JPEG images are allowed")

    contents = await file.read()
    if len(contents) > MAX_SIZE:
        raise HTTPException(413, "File too large (max 2 MB)")

    with open(f"uploads/{file.filename}", "wb") as f:   # create the folder first
        f.write(contents)

    return {"filename": file.filename, "size": len(contents), "type": file.content_type}


@app.post("/upload-many")
async def upload_many(files: list[UploadFile]):
    return {"filenames": [f.filename for f in files]}


@app.post("/upload-with-data")
async def upload_with_data(title: Annotated[str, Form()], file: UploadFile):
    return {"title": title, "filename": file.filename}
```

UploadFile attribute / method	Meaning
`file.filename`	Original file name, e.g. photo.png
`file.content_type`	The MIME type, e.g. image/png

UploadFile attribute / method	Meaning
`file.size`	Size in bytes (when known)
`await file.read()`	Read the whole file as bytes
`await file.seek(0)`	Go back to the start of the file
`file.file`	The underlying Python file object (for streaming/copying)

WARNING Never trust `file.filename` blindly; it is chosen by the user. In real apps, generate your own safe name (for example with `uuid.uuid4()`) to avoid overwriting files or path tricks like `../../`.

CHAPTER 14

14. Dependency Injection

Dependency Injection sounds scary, but it's one of FastAPI's best and simplest features. A **dependency** is just a function that FastAPI runs **before** your endpoint, giving the result to your endpoint.

Think of it like a **coffee shop**: before the barista (your endpoint) makes your coffee, someone else checks your loyalty card and takes the payment (the dependencies). The barista just receives "payment OK, here's the customer".

A first dependency

PYTHON

```
from typing import Annotated
from fastapi import Depends, FastAPI

app = FastAPI()


def pagination(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}


@app.get("/books")
def list_books(page: Annotated[dict, Depends(pagination)]):
    return {"books": "...", "pagination": page}


@app.get("/users")
def list_users(page: Annotated[dict, Depends(pagination)]):
    return {"users": "...", "pagination": page}
```

Both endpoints now accept `skip` and `limit` query parameters, but you wrote the logic **once**. Dependencies can use path, query, header, cookie and body parameters, exactly like endpoints.

Make it reusable with a type alias

PYTHON

```
PageDep = Annotated[dict, Depends(pagination)]


@app.get("/orders")
def list_orders(page: PageDep):        # short and clean!
    return {"pagination": page}
```

What dependencies are used for

- **Sharing logic**: pagination, filters, common parameters
- **Database sessions**: open a connection, give it to the endpoint, close it after
- **Security**: check the token, load the current user, check permissions
- **Configuration**: give endpoints the app settings
- **Testing**: swap a real dependency for a fake one (Chapter 24)

Classes as dependencies

```python
class Pagination:
    def __init__(self, skip: int = 0, limit: int = 10):
        self.skip = skip
        self.limit = limit


@app.get("/products")
def list_products(page: Annotated[Pagination, Depends()]):
    return {"skip": page.skip, "limit": page.limit}    # autocomplete works!
```

TIP With a class, you can write `Depends()` with nothing inside; FastAPI uses the type hint.

Dependencies that depend on other dependencies

```python
from fastapi import Header, HTTPException


def get_api_key(x_api_key: Annotated[str | None, Header()] = None) -> str:
    if x_api_key is None:
        raise HTTPException(status_code=401, detail="API key missing")
    return x_api_key


def get_client(api_key: Annotated[str, Depends(get_api_key)]) -> dict:
    clients = {"abc123": {"name": "Mobile App", "plan": "pro"}}
    client = clients.get(api_key)
    if not client:
        raise HTTPException(status_code=403, detail="Invalid API key")
    return client


@app.get("/reports")
def get_reports(client: Annotated[dict, Depends(get_client)]):
    return {"hello": client["name"]}
```

FastAPI builds the chain for you: `get_api_key` runs first, then `get_client`, then the endpoint. If the same dependency is used twice in one request, it runs only once (the result is cached).

Dependencies with yield (setup and cleanup)

Use `yield` when something must be **cleaned up** after the request, like closing a database connection:

```python
def get_db():
    db = connect_to_database()     # 1. setup: runs before the endpoint
    try:
        yield db                   # 2. the endpoint uses db here
    finally:
        db.close()                 # 3. cleanup: runs after the response
```

Dependencies on a whole group of endpoints

PYTHON

```python
def verify_token(x_token: Annotated[str, Header()]):
    if x_token != "secret-token":
        raise HTTPException(status_code=400, detail="Invalid X-Token header")


# For one endpoint where you don't need the return value:
@app.get("/admin/stats", dependencies=[Depends(verify_token)])
def admin_stats():
    return {"users": 120}

# For the WHOLE app:
# app = FastAPI(dependencies=[Depends(verify_token)])
# For a group of routes (see Chapter 18):
# router = APIRouter(dependencies=[Depends(verify_token)])
```

CHAPTER 15

15. Async vs Sync: def or async def?

This is the question beginners ask most. Here is the simple explanation.

The burger shop analogy

- **Sync (`def`)**: you order a burger and stand at the counter, doing nothing, until it's ready. The cashier can't serve anyone else.
- **Async (`async def`)**: you order, get a number, and sit down. The cashier serves other customers while your burger cooks. When it's ready, they call your number.

Async is great when your code spends a lot of time **waiting**: for a database, another API, a file, the network. While one request waits, the server handles others.

The simple rule

Your code inside the function...	Use
uses libraries that support `await` (httpx.AsyncClient, asyncpg, motor, aiofiles...)	`async def`
uses normal libraries that block (requests, most SQL libraries, time.sleep, file open...)	`def`
does not wait for anything (just computes and returns)	either (`async def` is slightly faster)
you are not sure	`def` (it is always safe)

PYTHON

```
import httpx
from fastapi import FastAPI

app = FastAPI()


# Async library -> async def + await
@app.get("/weather")
async def weather():
    async with httpx.AsyncClient() as client:
        r = await client.get("https://api.example.com/weather?city=Colombo")
    return r.json()


# Blocking library -> plain def (FastAPI runs it in a thread pool for you)
@app.get("/report")
def report():
    import time
    time.sleep(2)            # pretend this is slow, blocking work
    return {"status": "done"}
```

WARNING The worst mistake: calling a **blocking** function (like `requests.get()` or `time.sleep()`) inside an `async def`. It freezes the whole server for every user until it finishes. If you need a blocking call, use plain `def`.

NOTE When you use plain `def`, FastAPI runs your function in a separate thread, so it does not block other requests. That's why `def` is always safe. Heavy CPU work (image processing, machine learning) is better done in a separate worker process or task queue.

CHAPTER 16

16. Working with a Database (SQLModel)

Real applications store data in a **database** so it survives restarts. FastAPI works with any database. In this chapter we use **SQLModel**, a library made by the creator of FastAPI. It combines **SQLAlchemy** (the most popular Python database library) with **Pydantic**, so one class can be both a database table and a data model.

TERMINAL
```
$ pip install sqlmodel
```

We will use **SQLite**, a database stored in a single file. It needs no installation, which makes it perfect for learning. Later you can switch to PostgreSQL or MySQL by changing just one line (the URL).

Project 2: A Book Library API with a database

PYTHON
```
from contextlib import asynccontextmanager
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


# ---------- 1. Models ----------
class BookBase(SQLModel):
    title: str = Field(index=True)
    author: str
    year: int | None = None


class Book(BookBase, table=True):          # table=True -> a real DB table
    id: int | None = Field(default=None, primary_key=True)


class BookCreate(BookBase):                # what the client sends
    pass


class BookPublic(BookBase):                # what we send back
    id: int


class BookUpdate(SQLModel):                # every field optional
    title: str | None = None
    author: str | None = None
    year: int | None = None


# ---------- 2. Database connection ----------
sqlite_url = "sqlite:///books.db"
engine = create_engine(sqlite_url, connect_args={"check_same_thread": False})


def get_session():
    with Session(engine) as session:
        yield session                      # FastAPI closes it afterwards
```

```
SessionDep = Annotated[Session, Depends(get_session)]


# ---------- 3. Create tables when the app starts ----------
@asynccontextmanager
async def lifespan(app: FastAPI):
    SQLModel.metadata.create_all(engine)
    yield


app = FastAPI(lifespan=lifespan)


# ---------- 4. CRUD endpoints ----------
@app.post("/books", response_model=BookPublic, status_code=201)
def create_book(book: BookCreate, session: SessionDep):
    db_book = Book.model_validate(book)
    session.add(db_book)
    session.commit()
    session.refresh(db_book)              # load the new id from the DB
    return db_book


@app.get("/books", response_model=list[BookPublic])
def read_books(
    session: SessionDep,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 20,
):
    books = session.exec(select(Book).offset(offset).limit(limit)).all()
    return books


@app.get("/books/{book_id}", response_model=BookPublic)
def read_book(book_id: int, session: SessionDep):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    return book


@app.patch("/books/{book_id}", response_model=BookPublic)
def update_book(book_id: int, book: BookUpdate, session: SessionDep):
    db_book = session.get(Book, book_id)
    if not db_book:
        raise HTTPException(status_code=404, detail="Book not found")
    db_book.sqlmodel_update(book.model_dump(exclude_unset=True))
    session.add(db_book)
    session.commit()
    session.refresh(db_book)
    return db_book


@app.delete("/books/{book_id}")
def delete_book(book_id: int, session: SessionDep):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    session.delete(book)
    session.commit()
```

```
    return {"ok": True}
```

Understanding the code

1. Several models for one table

We use a family of models. This is a very common and recommended pattern:

Model	Purpose
`BookBase`	Shared fields, so we don't repeat ourselves
`Book` (`table=True`)	The real database table, includes the `id` primary key
`BookCreate`	What a client may send when creating (no `id`: the database creates it)
`BookPublic`	What we return (`id` is required here)
`BookUpdate`	All fields optional, for partial updates (PATCH)

Why not just one model? Because different operations need different fields. For example, a client must never be able to choose the `id`, and for a user table you would never return the password.

2. Engine and session

- The **engine** is the connection to the database. Create it **once** for the whole app.
- A **session** is a short conversation with the database, used for one request. `get_session` opens it, gives it to the endpoint with `yield`, and closes it afterwards.
- `SessionDep` is a type alias, so every endpoint just writes `session: SessionDep`.

3. Lifespan

The `lifespan` function runs code when the app **starts** (before `yield`) and **stops** (after `yield`). Here we create the tables at start-up. More on lifespan in Chapter 20.

4. The CRUD operations

Action	SQLModel code
Add a row	`session.add(obj)` then `session.commit()`
Reload from DB (get new id)	`session.refresh(obj)`
Get by primary key	`session.get(Book, book_id)`
Query many	`session.exec(select(Book)).all()`
Filter	`select(Book).where(Book.author == "Austen")`
Order	`select(Book).order_by(Book.year)`
Paginate	`select(Book).offset(20).limit(10)`
First match	`session.exec(select(Book).where(...)).first()`
Update fields	`obj.sqlmodel_update(data)` then add + commit
Delete	`session.delete(obj)` then `session.commit()`

Searching and filtering

PYTHON

```python
@app.get("/books/search/", response_model=list[BookPublic])
def search_books(
    session: SessionDep,
    author: str | None = None,
    min_year: int | None = None,
):
    query = select(Book)
    if author:
        query = query.where(Book.author.contains(author))
    if min_year:
        query = query.where(Book.year >= min_year)
    return session.exec(query.order_by(Book.title)).all()
```

Using another database

Database	URL example	Extra package
SQLite	`sqlite:///books.db`	none (built into Python)
PostgreSQL	`postgresql+psycopg://user:pass@localhost:5432/mydb`	`psycopg[binary]`
MySQL	`mysql+pymysql://user:pass@localhost:3306/mydb`	`pymysql`

NOTE Remove `connect_args={"check_same_thread": False}` for databases other than SQLite; it is an SQLite-only setting.

Database migrations with Alembic

`create_all()` creates tables that do not exist, but it **does not change** existing tables when you add a column. In real projects, use **Alembic** to manage changes to your database structure (called *migrations*):

TERMINAL

```
$ pip install alembic
$ alembic init migrations                                  # set up once
$ alembic revision --autogenerate -m "add books table"  # after changing models
$ alembic upgrade head                                     # apply changes
```

TIP Other popular choices: plain **SQLAlchemy 2.0** (more control), **Tortoise ORM** (async), and **Beanie** or **Motor** for MongoDB. The FastAPI patterns (dependency with yield, response models) stay the same.

CHAPTER 17

17. Authentication and Security (JWT)

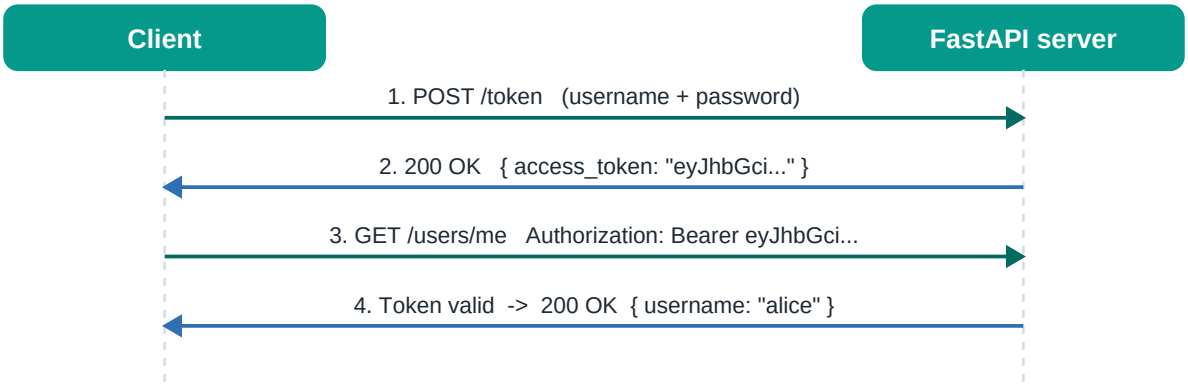
Most APIs need users to **log in**. Two words to know:

- **Authentication**: *who are you?* (checking username and password)
- **Authorization**: *what are you allowed to do?* (checking permissions)

How token login works

1. The user sends their **username and password** to `/token`.
2. The server checks them. If correct, it creates a **token**: a signed string, like a cinema ticket.
3. The client stores the token and sends it with every request in a header: `Authorization: Bearer <token>`.
4. For protected endpoints, the server checks the token's signature and expiry, then knows who the user is. No password needed again.

We use **JWT** (JSON Web Token), the most common token format. A JWT contains data (like the username and expiry time) and a **signature** made with your secret key, so nobody can change it without being detected.



Install the libraries

TERMINAL
```
$ pip install pyjwt "pwdlib[argon2]"
```

- `pyjwt` creates and checks JWT tokens.
- `pwdlib` with **Argon2** hashes passwords securely.

WARNING **Never store passwords as plain text.** Always store a *hash*: a one-way scrambled version. When the user logs in, you hash what they typed and compare the hashes. If your database leaks, the real passwords stay safe.

Project 3: Login with JWT

PYTHON

```python
from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt
from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jwt.exceptions import InvalidTokenError
from pwdlib import PasswordHash
from pydantic import BaseModel

# ---------- Settings (keep the secret key out of your code in real apps!) ----------
SECRET_KEY = "change-me-to-a-long-random-secret-key-0123456789"  # openssl rand -hex 32
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

password_hash = PasswordHash.recommended()     # uses Argon2
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# A fake database with one user. Password is "secret123".
fake_users_db = {
    "alice": {
        "username": "alice",
        "full_name": "Alice Perera",
        "hashed_password": password_hash.hash("secret123"),
        "disabled": False,
    }
}


# ---------- Models ----------
class Token(BaseModel):
    access_token: str
    token_type: str


class User(BaseModel):
    username: str
    full_name: str | None = None
    disabled: bool = False


class UserInDB(User):
    hashed_password: str


# ---------- Helper functions ----------
def get_user(username: str) -> UserInDB | None:
    if username in fake_users_db:
        return UserInDB(**fake_users_db[username])
    return None


def authenticate_user(username: str, password: str) -> UserInDB | None:
    user = get_user(username)
    if not user or not password_hash.verify(password, user.hashed_password):
        return None
    return user
```

```
def create_access_token(data: dict, expires_delta: timedelta) -> str:
    to_encode = data.copy()
    to_encode["exp"] = datetime.now(timezone.utc) + expires_delta
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)


# ---------- Dependencies ----------
async def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]) -> User:
    credentials_error = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise credentials_error
    except InvalidTokenError:
        raise credentials_error
    user = get_user(username)
    if user is None:
        raise credentials_error
    return user


async def get_current_active_user(
    current_user: Annotated[User, Depends(get_current_user)],
) -> User:
    if current_user.disabled:
        raise HTTPException(status_code=400, detail="Inactive user")
    return current_user


app = FastAPI()


# ---------- Endpoints ----------
@app.post("/token")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    user = authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    token = create_access_token(
        data={"sub": user.username},
        expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
    )
    return Token(access_token=token, token_type="bearer")


@app.get("/users/me", response_model=User)
async def read_me(current_user: Annotated[User, Depends(get_current_active_user)]):
    return current_user
```

Try it in /docs

1. Run the app and open /docs. Notice the **Authorize** button with a lock icon at the top.
2. Try **GET /users/me** first: you get `401 Not authenticated`.
3. Click **Authorize**, enter username `alice` and password `secret123`, and click Authorize.
4. Try **GET /users/me** again. Now it returns Alice's data! The docs page sends the token for you.

How the pieces fit

Piece	Job
`OAuth2PasswordBearer(tokenUrl="token")`	Reads the `Authorization: Bearer ...` header and tells /docs where to log in
`OAuth2PasswordRequestForm`	Reads `username` and `password` from the login form
`password_hash.hash()` / `.verify()`	Create a hash / check a password against a hash
`create_access_token()`	Builds a JWT with the username (`sub`) and expiry (`exp`)
`get_current_user`	Dependency: decodes the token and loads the user, or raises 401
`get_current_active_user`	Dependency on a dependency: also checks the user isn't disabled

Protecting any endpoint

Now protecting an endpoint is one line. Just add the dependency:

```python
CurrentUser = Annotated[User, Depends(get_current_active_user)]


@app.get("/my-orders")
async def my_orders(user: CurrentUser):
    return {"owner": user.username, "orders": []}
```

Simple role checking (authorization)

```python
def require_admin(user: CurrentUser) -> User:
    if user.username != "admin":            # in real apps, check a role field
        raise HTTPException(status_code=403, detail="Admins only")
    return user


@app.delete("/users/{username}", dependencies=[Depends(require_admin)])
async def delete_user(username: str):
    return {"deleted": username}
```

Security checklist

- Generate a strong secret key (`openssl rand -hex 32`) and load it from an **environment variable**, never commit it to Git.

- Always use **HTTPS** in production, so tokens and passwords can't be read in transit.
- Keep access tokens **short-lived** (15 to 60 minutes). Use refresh tokens for longer sessions.
- Return the same error for "wrong username" and "wrong password", so attackers can't discover usernames.
- Add **rate limiting** to the login endpoint (for example with the `slowapi` package) to slow down password guessing.
- Validate all input (FastAPI helps!) and never build SQL strings by hand; the ORM protects against SQL injection.

> **TIP** For simple machine-to-machine access, an **API key** in a header (see the dependency example in Chapter 14) is often enough. For "Login with Google/GitHub", look at OAuth2 libraries such as **Authlib**.

CHAPTER 18

18. Structuring Bigger Applications (APIRouter)

Putting everything in one `main.py` works for small apps, but quickly becomes messy. **APIRouter** lets you split your API into several files, like chapters in a book.

A recommended project structure

PROJECT LAYOUT

```
my-project/
|-- app/
|   |-- __init__.py          # empty file: makes "app" a Python package
|   |-- main.py              # creates FastAPI() and includes the routers
|   |-- config.py            # settings (Chapter 21)
|   |-- database.py          # engine + get_session
|   |-- dependencies.py      # shared dependencies (current user, etc.)
|   |-- models/              # database / Pydantic models
|   |   |-- __init__.py
|   |   |-- book.py
|   |   +-- user.py
|   |-- routers/             # one file per topic
|   |   |-- __init__.py
|   |   |-- books.py
|   |   +-- users.py
|   +-- services/            # business logic (optional, for bigger apps)
|-- tests/
|   +-- test_books.py
|-- .env                     # secrets (never commit!)
|-- requirements.txt
+-- Dockerfile
```

A router file: app/routers/books.py

PYTHON

```python
from fastapi import APIRouter, HTTPException

router = APIRouter(
    prefix="/books",          # every path in this file starts with /books
    tags=["Books"],           # groups these endpoints in /docs
    responses={404: {"description": "Not found"}},
)

fake_books = {1: {"title": "Dune"}}


@router.get("/")              # -> GET /books/
def list_books():
    return list(fake_books.values())


@router.get("/{book_id}")     # -> GET /books/{book_id}
def get_book(book_id: int):
    if book_id not in fake_books:
        raise HTTPException(status_code=404, detail="Book not found")
    return fake_books[book_id]
```

Notice: you use `@router.get` instead of `@app.get`. Everything else is identical.

Another router: app/routers/users.py

PYTHON

```python
from fastapi import APIRouter

router = APIRouter(prefix="/users", tags=["Users"])


@router.get("/")
def list_users():
    return [{"username": "alice"}]
```

Connect them: app/main.py

PYTHON

```python
from fastapi import FastAPI

from app.routers import books, users

app = FastAPI(title="Library API")

app.include_router(books.router)
app.include_router(users.router)
# Add a version prefix to everything if you like:
# app.include_router(books.router, prefix="/api/v1")


@app.get("/")
def root():
    return {"message": "Welcome to the Library API"}
```

TERMINAL

```
# run from the my-project folder
$ fastapi dev app/main.py
```

TIP Keep endpoint functions thin: receive data, call a service function, return the result. Put business rules in a separate `services` module. This makes code easier to test and reuse.

CHAPTER 19

19. Middleware and CORS

What is middleware?

Middleware is code that runs for **every** request, before it reaches your endpoint, and for every response, before it goes back to the client. It's like a security guard at the building entrance who checks everyone going in and out.

PYTHON

```
import time
from fastapi import FastAPI, Request

app = FastAPI()


@app.middleware("http")
async def add_process_time(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)          # run the endpoint
    duration = time.perf_counter() - start
    response.headers["X-Process-Time"] = f"{duration:.4f}"
    print(f"{request.method} {request.url.path} -> {response.status_code} in {duration:.4f}s"
)
    return response
```

Common uses: logging, timing, adding headers, request IDs, compression.

CORS: letting websites call your API

Browsers have a safety rule: a web page from one address (like `http://localhost:3000`, your React or Vue app) is **not allowed** to call an API at a different address (like `http://localhost:8000`) unless the API says it's OK. This is called **CORS** (Cross-Origin Resource Sharing). If you see a CORS error in the browser console, this is the fix:

PYTHON

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",          # your frontend during development
        "https://www.mywebsite.com",      # your real website
    ],
    allow_credentials=True,               # allow cookies / Authorization header
    allow_methods=["*"],                  # allow all HTTP methods
    allow_headers=["*"],                  # allow all headers
)
```

WARNING Avoid `allow_origins=["*"]` (any website) in production, especially with `allow_credentials=True`. List your real frontend addresses instead.

> **NOTE** CORS is only enforced by **browsers**. Mobile apps, Postman, curl and other servers are not affected by it.

Other built-in middleware

Middleware	What it does
`GZipMiddleware`	Compresses large responses so they download faster
`TrustedHostMiddleware`	Rejects requests with an unexpected Host header
`HTTPSRedirectMiddleware`	Redirects all http:// requests to https://

PYTHON
```
from fastapi.middleware.gzip import GZipMiddleware

app.add_middleware(GZipMiddleware, minimum_size=1000)
```

CHAPTER 20

20. Background Tasks and Lifespan Events

Background tasks

Sometimes you want to do something **after** sending the response, so the user doesn't have to wait: sending a welcome email, writing a log, resizing an image. Use `BackgroundTasks`:

PYTHON

```
from fastapi import BackgroundTasks, FastAPI

app = FastAPI()


def send_welcome_email(email: str, name: str):
    # imagine slow email-sending code here
    with open("emails.log", "a") as f:
        f.write(f"Sent welcome email to {name} <{email}>\n")


@app.post("/register")
def register(email: str, name: str, background_tasks: BackgroundTasks):
    # ... save the user ...
    background_tasks.add_task(send_welcome_email, email, name)
    return {"message": "Registered! Check your inbox soon."}   # returned immediately
```

NOTE Background tasks are perfect for small jobs. For heavy or important jobs (that must not be lost if the server restarts, or that take minutes), use a task queue like **Celery**, **RQ**, **Dramatiq** or **ARQ** with Redis.

Lifespan: start-up and shut-down code

Some things should happen **once** when the app starts (load a machine learning model, connect to a database, warm a cache) and be cleaned up when it stops. Use a **lifespan** function:

PYTHON

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

ml_models = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    # ---- runs ONCE at start-up ----
    print("Starting: loading model...")
    ml_models["sentiment"] = lambda text: "positive" if "good" in text else "negative"
    yield                                   # the app runs and serves requests here
    # ---- runs ONCE at shut-down ----
    print("Shutting down: cleaning up...")
    ml_models.clear()


app = FastAPI(lifespan=lifespan)
```

```
@app.get("/predict")
def predict(text: str):
    return {"sentiment": ml_models["sentiment"](text)}
```

NOTE Older tutorials use `@app.on_event("startup")` and `@app.on_event("shutdown")`. They are deprecated; use lifespan instead.

CHAPTER 21

21. Settings and Environment Variables

Secrets (passwords, API keys) and settings that change between your laptop and the server (database URL, debug mode) should **not** be written in your code. Put them in **environment variables**, usually stored in a `.env` file during development.

TERMINAL
```
$ pip install pydantic-settings
```

The .env file

.ENV
```
APP_NAME="Library API"
DEBUG=true
DATABASE_URL=sqlite:///books.db
SECRET_KEY=3f6a9c2e7b1d4f8a0c5e9b2d6f1a4c7e8b0d3f6a9c2e5b8d1f4a7c0e3b6d9f2a
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

WARNING Add `.env` to your `.gitignore` file so secrets are never uploaded to GitHub. Commit a `.env.example` file with fake values instead, so others know which variables are needed.

The settings class: app/config.py

PYTHON
```
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    app_name: str = "My API"
    debug: bool = False
    database_url: str
    secret_key: str
    access_token_expire_minutes: int = 30

    model_config = SettingsConfigDict(env_file=".env")


@lru_cache          # read the settings only once, then reuse them
def get_settings() -> Settings:
    return Settings()
```

Pydantic Settings reads values from environment variables (and the `.env` file), converts them to the right types, and gives a clear error at start-up if a required value is missing. Names are matched without caring about upper/lower case.

Using the settings

PYTHON
```
from typing import Annotated
from fastapi import Depends, FastAPI
```

```
from app.config import Settings, get_settings

app = FastAPI()


@app.get("/info")
def info(settings: Annotated[Settings, Depends(get_settings)]):
    return {"app_name": settings.app_name, "debug": settings.debug}
```

TIP Using settings as a dependency means tests can easily replace them with test settings (see Chapter 24).

CHAPTER 22

22. WebSockets, Static Files and Templates

WebSockets: real-time two-way communication

Normal HTTP is like sending letters: the client asks, the server answers, done. A **WebSocket** is like a phone call: the connection stays open and **both sides can send messages at any time**. Use it for chat apps, live notifications, multiplayer games and live dashboards.

PYTHON

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()


class ConnectionManager:
    def __init__(self):
        self.active: list[WebSocket] = []

    async def connect(self, ws: WebSocket):
        await ws.accept()
        self.active.append(ws)

    def disconnect(self, ws: WebSocket):
        self.active.remove(ws)

    async def broadcast(self, message: str):
        for ws in self.active:
            await ws.send_text(message)


manager = ConnectionManager()


@app.websocket("/ws/chat/{username}")
async def chat(websocket: WebSocket, username: str):
    await manager.connect(websocket)
    await manager.broadcast(f"{username} joined the chat")
    try:
        while True:
            text = await websocket.receive_text()        # wait for a message
            await manager.broadcast(f"{username}: {text}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"{username} left the chat")
```

In the browser, JavaScript connects like this:

JAVASCRIPT

```
const ws = new WebSocket("ws://127.0.0.1:8000/ws/chat/nirodha");
ws.onmessage = (event) => console.log(event.data);
ws.send("Hello everyone!");
```

NOTE This simple manager keeps connections in memory, so it works with one server process. For many servers, use a message broker such as Redis pub/sub to share messages between them.

Serving static files

To serve images, CSS and JavaScript files from a folder, **mount** a `StaticFiles` app:

PYTHON

```python
from fastapi.staticfiles import StaticFiles

app.mount("/static", StaticFiles(directory="static"), name="static")
# static/logo.png is now available at http://127.0.0.1:8000/static/logo.png
```

HTML pages with Jinja2 templates

FastAPI is mostly used for APIs, but it can also render web pages:

PYTHON

```python
from fastapi import Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="templates")


@app.get("/books/{book_id}/page", response_class=HTMLResponse)
def book_page(request: Request, book_id: int):
    book = {"id": book_id, "title": "Dune", "author": "Frank Herbert"}
    return templates.TemplateResponse(request, "book.html", {"book": book})
```

HTML

```html
<!-- templates/book.html -->
<html>
  <head><link href="/static/style.css" rel="stylesheet"></head>
  <body>
    <h1>{{ book.title }}</h1>
    <p>Written by {{ book.author }}</p>
  </body>
</html>
```

CHAPTER 23

23. Customizing the API Documentation

Your /docs page is often the first thing other developers see. Make it great:

```python
from fastapi import FastAPI

tags_metadata = [
    {"name": "Books", "description": "Manage the books in the library."},
    {"name": "Users", "description": "Sign-up, login and profiles."},
]

app = FastAPI(
    title="Library API",
    summary="A friendly API for a small library.",
    description="""
Use this API to **search**, **add** and **borrow** books.

* Books: create, read, update, delete
* Users: register and log in
""",
    version="1.2.0",
    contact={"name": "Support", "email": "support@example.com"},
    license_info={"name": "MIT"},
    openapi_tags=tags_metadata,
)
```

Documenting each endpoint

```python
@app.post(
    "/books",
    tags=["Books"],
    summary="Add a new book",
    response_description="The newly created book",
    status_code=201,
)
def create_book(book: Book):
    """
    Create a book with all its information:

    - **title**: the book's title (required)
    - **author**: who wrote it (required)
    - **pages**: number of pages
    """
    return book


@app.get("/books/old-search", tags=["Books"], deprecated=True)
def old_search():
    """Use /books/search instead."""
    return []
```

The function's **docstring** (the text in triple quotes) appears in /docs, and it supports Markdown formatting.

Setting	Example	Effect
Change docs URL	`FastAPI(docs_url="/documentation")`	Swagger UI at a new address
Disable docs	`FastAPI(docs_url=None, redoc_url=None)`	Hide the docs pages (e.g. in production)
Hide one endpoint	`@app.get("/x", include_in_schema=False)`	Endpoint works but isn't documented

CHAPTER 24

24. Testing Your API

Tests are small programs that check your API works correctly. When you change code later, you run the tests to make sure you didn't break anything. FastAPI makes testing easy with `TestClient`, which calls your app directly without starting a server.

TERMINAL
```
$ pip install pytest
```

Your first test

Suppose this is **main.py**:

PYTHON
```
from fastapi import FastAPI, HTTPException

app = FastAPI()
books = {1: {"title": "Dune"}}


@app.get("/books/{book_id}")
def get_book(book_id: int):
    if book_id not in books:
        raise HTTPException(status_code=404, detail="Book not found")
    return books[book_id]
```

Create **test_main.py** next to it:

PYTHON
```
from fastapi.testclient import TestClient

from main import app

client = TestClient(app)


def test_get_existing_book():
    response = client.get("/books/1")
    assert response.status_code == 200
    assert response.json() == {"title": "Dune"}


def test_get_missing_book():
    response = client.get("/books/999")
    assert response.status_code == 404
    assert response.json() == {"detail": "Book not found"}


def test_invalid_id():
    response = client.get("/books/abc")
    assert response.status_code == 422
```

TERMINAL
```
$ pytest              # finds and runs every test_*.py file
$ pytest -v           # verbose: shows each test name
```

Each `assert` checks that something is true. If any assert fails, pytest shows you exactly what went wrong.

Sending data in tests

PYTHON

```
def test_create_todo():
    response = client.post("/todos", json={"title": "Write tests"})
    assert response.status_code == 201
    data = response.json()
    assert data["title"] == "Write tests"
    assert "id" in data

# Other useful options:
# client.get("/items", params={"skip": 0, "limit": 5})        # query parameters
# client.get("/users/me", headers={"Authorization": "Bearer x"})
# client.post("/token", data={"username": "a", "password": "b"})   # form data
# client.post("/upload", files={"file": ("a.png", b"...", "image/png")})
```

Overriding dependencies (fake database, fake user)

This is where dependency injection really shines. In tests, you can replace any dependency with a fake one:

PYTHON

```
from main import app, get_current_user


def fake_current_user():
    return {"username": "test-user"}


app.dependency_overrides[get_current_user] = fake_current_user   # swap it in

# ... run tests: every endpoint now sees "test-user" as logged in ...

app.dependency_overrides.clear()                                  # restore
```

TIP For database tests, override `get_session` with a session connected to a separate test database (for example an in-memory SQLite database `sqlite://`), so tests never touch your real data.

NOTE To run lifespan start-up code during tests, use the client as a context manager: `with TestClient(app) as client:`. For fully async tests, look at `httpx.AsyncClient` with the `anyio` pytest plugin.

CHAPTER 25

25. Deployment: Putting Your API Online

Deployment means running your API on a server so other people can use it over the internet.

Development vs production

	`fastapi dev`	`fastapi run`
Purpose	Writing code on your computer	Running for real users
Auto-reload	Yes	No
Listens on	127.0.0.1 (only your computer)	0.0.0.0 (all network interfaces)

TERMINAL
```
$ fastapi run main.py --port 8000 --workers 4
```

`--workers 4` starts 4 copies of your app so it can use 4 CPU cores. A common starting point is one worker per CPU core. The equivalent Uvicorn command is:

TERMINAL
```
$ uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

Deploying with Docker

Docker packages your app with everything it needs (Python, libraries, code) into a **container** that runs the same everywhere. Create a file named **Dockerfile** (no extension):

DOCKERFILE
```
FROM python:3.13-slim

WORKDIR /code

# Install dependencies first (Docker caches this step, so rebuilds are fast)
COPY ./requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

# Copy your application code
COPY ./app /code/app

# Start the server
CMD ["fastapi", "run", "app/main.py", "--port", "80"]
```

TERMINAL
```
$ docker build -t library-api .                        # build the image
$ docker run -d -p 8000:80 --env-file .env library-api  # run it
```

Your API is now at http://localhost:8000. Also create a **.dockerignore** file containing `.venv`, `__pycache__`, `.env` and `.git`, so these are not copied into the image.

Running behind a proxy (HTTPS)

In production, a **reverse proxy** such as **Nginx**, **Caddy** or **Traefik** usually sits in front of your app. It handles HTTPS certificates (free from Let's Encrypt), and forwards requests to FastAPI. Tell FastAPI to trust the proxy's headers:

TERMINAL

```
$ fastapi run app/main.py --proxy-headers --forwarded-allow-ips="*"
```

NOTE Only use `--forwarded-allow-ips="*"` when the app is reachable **only** through your proxy (for example inside a private Docker network). Otherwise list the proxy's IP address.

Where to host

Type	Examples	Good for
Platform-as-a-Service	Render, Railway, Fly.io, Heroku, FastAPI Cloud	Easiest start: connect GitHub and deploy
Cloud containers	Google Cloud Run, AWS App Runner / ECS, Azure Container Apps	Scaling Docker containers
Virtual server (VPS)	DigitalOcean, Linode, AWS EC2, Hetzner	Full control (you manage everything)
Kubernetes	GKE, EKS, AKS	Large systems with many services

Production checklist

- Secrets come from environment variables, not code. `DEBUG` is off.
- HTTPS is enabled. CORS lists only your real frontend addresses.
- Use a real database (PostgreSQL is a great default) with Alembic migrations and backups.
- Logs are collected. Errors are monitored (for example with Sentry).
- Add a `/health` endpoint that returns `{"status": "ok"}` so your host can check the app is alive.
- Run several workers or replicas. Put limits on upload sizes and request rates.
- Consider hiding /docs in production if the API is private.

CHAPTER 26

26. Best Practices and Common Mistakes

Best practices

1. **Use type hints everywhere.** They power validation, documentation and editor help.
2. **Separate input and output models** (`UserCreate`, `UserPublic`). Never return passwords or internal fields.
3. **Always set a** `response_model` or return type for endpoints that return data.
4. **Use the right status codes**: 201 for create, 204 for delete, 404 for not found.
5. **Use dependencies** for anything repeated: database sessions, current user, pagination, settings.
6. **Split the app with APIRouter** once it grows beyond one or two topics.
7. **Keep secrets in environment variables** with pydantic-settings.
8. **Validate everything**: lengths, ranges, formats. Let FastAPI reject bad data early.
9. **Paginate list endpoints** and set a maximum `limit` so nobody can request a million rows.
10. **Write tests** with TestClient and run them before every deployment.
11. **Version your API** (`/api/v1/...`) so you can change it later without breaking existing clients.
12. **Use a code formatter and linter** such as **Ruff** to keep code clean and consistent.

Common mistakes and their fixes

Mistake	What happens	Fix
Blocking call (`requests`, `time.sleep`) inside `async def`	The whole server freezes	Use `def`, or an async library with `await`
`/users/{id}` declared before `/users/me`	"me" is treated as an id	Declare fixed paths first
Returning the database user model directly	Password hash leaks to clients	Use a `response_model` without secrets
Forgetting `session.commit()`	Data is not saved	Commit after add / update / delete
Using `.dict()` / `.json()`	Deprecation warnings	Use `.model_dump()` / `.model_dump_json()`
Mutable global variables to store data	Data lost on restart, not shared between workers	Use a database
`allow_origins=["*"]` in production	Any website can call your API from a browser	List your real frontends
Secret key written in the code	Anyone with the code can forge tokens	Load it from an environment variable
Using `fastapi dev` in production	Slow, auto-reload, only local	Use `fastapi run` with workers
Missing quotes: `pip install fastapi[standard]`	Error in zsh	Use quotes: `"fastapi[standard]"`

Troubleshooting quick guide

Error / symptom	Likely cause
`ModuleNotFoundError: No module named 'fastapi'`	Virtual environment not activated, or FastAPI not installed in it
`Address already in use`	Another server is using port 8000. Stop it or use `--port 8001`
`422 Unprocessable Content`	Request data doesn't match your model. Read the `detail` field
`405 Method Not Allowed`	Path exists but not for that method (e.g. GET instead of POST)
`404 Not Found` for an endpoint you wrote	Typo in the path, missing router prefix, or router not included
CORS error in browser console	Add `CORSMiddleware` with your frontend's address
`Form data requires "python-multipart"`	Install `python-multipart` (or `fastapi[standard]`)
Changes don't appear	You used `fastapi run` (no reload) or saved a different file

CHAPTER 27

27. Cheat Sheet

Commands

TERMINAL

```
$ python -m venv .venv                  # create a virtual environment
$ source .venv/bin/activate             # activate (Windows: .venv\Scripts\activate)
$ pip install "fastapi[standard]"       # install FastAPI with extras
$ fastapi dev main.py                   # run in development (auto-reload)
$ fastapi run main.py --workers 4       # run in production
$ uvicorn main:app --reload             # alternative way to run
$ pytest -v                             # run tests
```

Important URLs

URL	What you find
`http://127.0.0.1:8000/docs`	Interactive Swagger UI documentation
`http://127.0.0.1:8000/redoc`	ReDoc documentation
`http://127.0.0.1:8000/openapi.json`	The raw OpenAPI schema

Where does a parameter come from?

You write	FastAPI reads it from
`item_id: int` (and `{item_id}` is in the path)	Path
`q: str | None = None`	Query string
`item: Item` (a Pydantic model)	JSON body
`Annotated[str, Header()]`	Header
`Annotated[str, Cookie()]`	Cookie
`Annotated[str, Form()]`	Form data
`file: UploadFile`	Uploaded file
`Annotated[X, Depends(func)]`	The result of a dependency
`request: Request`	The raw request object
`background_tasks: BackgroundTasks`	Background task manager

The most useful imports

PYTHON

```
from typing import Annotated
from fastapi import (FastAPI, APIRouter, Depends, HTTPException, status, Request,
                     Response, Query, Path, Body, Header, Cookie, Form, File,
```

```
                     UploadFile, BackgroundTasks, WebSocket, WebSocketDisconnect)
from fastapi.responses import (JSONResponse, HTMLResponse, FileResponse,
                               RedirectResponse, StreamingResponse)
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field, EmailStr, field_validator
```

The endpoint template

PYTHON

```
@router.post("/things", response_model=ThingPublic, status_code=201, tags=["Things"])
def create_thing(
    thing: ThingCreate,                         # body
    session: SessionDep,                        # database
    user: CurrentUser,                          # logged-in user
    notify: bool = False,                       # query parameter
):
    """Short description shown in /docs."""
    db_thing = Thing.model_validate(thing, update={"owner": user.username})
    session.add(db_thing)
    session.commit()
    session.refresh(db_thing)
    return db_thing
```

CHAPTER 28

28. Glossary and What to Learn Next

Glossary

Term	Simple meaning
API	A way for programs to talk to each other
Endpoint / path operation	One URL + method combination, e.g. GET /books
Request / Response	The message the client sends / the message the server returns
HTTP method	The action: GET, POST, PUT, PATCH, DELETE
Status code	A number describing the result, e.g. 200 OK, 404 Not Found
JSON	A text format for data, similar to Python dictionaries
Path parameter	A variable part of the URL path: /books/{id}
Query parameter	Optional values after ? in the URL: ?page=2
Request body	Data sent by the client, usually JSON
Pydantic model	A class describing the shape and rules of data
Validation	Checking data follows the rules before using it
Serialization	Converting Python objects into JSON (and back)
Dependency injection	FastAPI runs helper functions and passes their results to you
Middleware	Code that runs around every request and response
ORM	A library that lets you use the database with Python classes instead of SQL
Migration	A controlled change to the database structure
Authentication / Authorization	Who are you? / What are you allowed to do?
JWT	A signed token that proves who the user is
Hashing	One-way scrambling, used to store passwords safely
CORS	Browser rule controlling which websites may call your API
ASGI	The modern standard that connects async Python web apps to servers like Uvicorn
Uvicorn	The ASGI web server that runs FastAPI apps
OpenAPI	A standard format describing an API; it powers /docs
Async	Code that can pause while waiting so other work can run
Worker	One running copy of your app process

Your learning path

1. **Practice the basics**: rebuild the To-Do API from memory without looking.
2. **Add a database**: convert the To-Do API to SQLModel + SQLite, then PostgreSQL.
3. **Add users**: registration, login with JWT, and make each user see only their own to-dos.
4. **Structure it**: split it into routers, add settings and tests.
5. **Ship it**: put it in Docker and deploy it to a free hosting platform.
6. **Connect a frontend**: build a small React, Vue or plain HTML page that uses your API.

Project ideas to practice

- **Personal expense tracker**: categories, monthly summaries, CSV export.
- **URL shortener**: create short codes, redirect, and count clicks.
- **Blog API**: posts, comments, tags, authors with login and roles.
- **Weather proxy**: call a public weather API with httpx async, cache results.
- **Real-time chat**: WebSockets with chat rooms and message history in a database.
- **Library system**: books, members, borrowing with due dates and background reminder emails.

Official resources

Resource	Address
FastAPI documentation (excellent, with a full tutorial)	fastapi.tiangolo.com
Pydantic documentation	docs.pydantic.dev
SQLModel documentation	sqlmodel.tiangolo.com
Full-stack FastAPI template (real project example)	github.com/fastapi/full-stack-fastapi-template
Uvicorn documentation	www.uvicorn.org

Congratulations on reaching the end! The best way to learn now is to **build something you care about**. Start small, use /docs to test as you go, and add one new feature at a time. Happy coding!